

Code for practical 6

```
package Practical6
import breeze.linalg._
object pract6 {
  def main (Args: Array[String]): Unit = {
    println("Shreyash Kadam S090")

    val S1 = DenseMatrix(
      (90, 94, 98, 102),
      (106, 110, 114, 118),
      (122, 126, 130, 134),
      (138, 142, 146, 150)
    )

    println(s"Original Matrix: \n$S1")

    val S2 = S1(1 to 3, 1 to 3)
    println(s"\nSlicing of matrix: \n$S2")

    val S3 = sum(S2(*, ::))
    println(s"\nSum of Rows: $S3")

    val S4 = sum(S2(:, *))
    println(s"\nSum of Columns: $S4")

  }
}
```

Output for practical 6

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" .
Shreyash Kadam S090
Original Matrix:
90  94  98  102
106 110 114 118
122 126 130 134
138 142 146 150

Slicing of matrix:
110 114 118
```

```
110 114 118
126 130 134
142 146 150
```

```
Sum of Rows: DenseVector(342, 390, 438)
```

```
Sum of Columns: Transpose(DenseVector(378, 390, 402))
```

```
Process finished with exit code 0
```

Code for practical 7

```
package practical7
import breeze.linalg._
```

```
object pract7 {
  def main(args: Array[String]): Unit = {
    println("Shreyash Kadam S090")

    val S1 = DenseMatrix((1.0, 2.0),(3.0, 4.0))
    println(s"\nMatrix 1:\n$S1")
    val S2 = DenseMatrix((5.0, 6.0),(7.0, 8.0))
    println(s"\nMatrix 2:\n$S2")
```

```
val S4 = S1 + S2
```

```

println(s"\nAddition of two Same Size Matrix:\n$S4")

val S5 = S2 - S1
println(s"\nSubstraction of two Same Size Matrix:\n$S5")

val S6 = S1 * S2
println(s"\nMultiplication of Two Same Size Matrix:\n$S6")

val S7 = S2 / S1
print(s"\nDivision of Two Same Size Matrix:\n$S7")

}

}

```

Output for practical 7

```

"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Shreyash Kadam S090

Matrix 1:
1.0  2.0
3.0  4.0

Matrix 2:
5.0  6.0
7.0  8.0

Addition of two Same Size Matrix:
6.0  8.0
10.0 12.0

Substraction of two Same Size Matrix:
4.0  4.0
4.0  4.0

```

Multiplication of Two Same Size Matrix:

19.0 22.0

43.0 50.0

Division of Two Same Size Matrix:

5.0 3.0

2.3333333333333335 2.0

Code for practical 8

```
import com.github.tototoshi.csv._
import java.io.File
import scala.util.Try
import scala.math._

object pract_8 {
  println("Shreyash Kadam S091")
  def main(args: Array[String]): Unit = {
    val reader = CSVReader.open(new File("heart.csv"))
    val allRows = reader.allWithHeaders()
    reader.close()
    val numericCols = allRows.head.keys.filter { col =>
      Try(allRows.head(col).toDouble).isSuccess
    }.toList
    println(f"\nNumeric Columns: ${numericCols.mkString(", ")}\n")
    for (col <- numericCols) {
      val values = allRows.flatMap(row =>
        Try(row(col).toDouble).toOption)
      val count = values.size
      val mean = values.sum / count
      val minVal = values.min
      val maxVal = values.max
      val stdDev = sqrt(values.map(x => pow(x - mean, 2)).sum / count)
      println(s"Column: $col")
      println(f" Count : $count")
      println(f" Mean : $mean%.2f")
      println(f" Min : $minVal%.2f")
      println(f" Max : $maxVal%.2f")
      println(f" Std Dev : $stdDev%.2f\n")
    }
  }
}
```

```
}  
}
```

Output for practical 8

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Users\Asus\OneDrive\Documents\Apps\Attachments\Desktop\Shre  
Shreyash Kadam S091  
  
Numeric Columns: oldpeak, trestbps, fbs, age, sex, restecg, slope, ca, thal, target, chol, exang, cp, thalach  
  
Column: oldpeak  
Count : 303  
Mean : 1.04  
Min : 0.00  
Max : 6.20  
Std Dev : 1.16  
  
Column: trestbps  
Count : 303  
Mean : 131.62  
Min : 94.00  
Max : 200.00  
Std Dev : 17.51  
  
Column: fbs  
Count : 303  
Mean : 0.15  
Min : 0.00  
Max : 1.00  
Std Dev : 0.36  
  
Column: age  
Count : 303  
Mean : 54.37  
Min : 29.00  
Max : 77.00  
Std Dev : 9.07
```

Code for practical 9

```
package practical9  
  
import com.github.tototoshi.csv._  
import java.io.File  
import scala.util.Try  
  
object practical9 {  
  def main(args: Array[String]): Unit = {  
    val inputFile = new File("all_stocks_5yr.csv")  
    val reader = CSVReader.open(inputFile)  
    val allRows = reader.allWithHeaders()  
    reader.close()  
    val numericColumns = Seq("open", "high")
```

```

val stats: Map[String, (Double, Int)] = numericColumns.map { col =>
  val values = allRows.map(row => row.getOrElse(col, "").trim)
  val validNumbers = values.flatMap(v => Try(v.toDouble).toOption)
  val missingCount = values.count(v => Try(v.toDouble).isFailure)
  val mean = if (validNumbers.nonEmpty) validNumbers.sum / validNumbers.size
  else 0.0
  (col, (mean, missingCount))
}.toMap

println("\n--- Missing Data Report ---")
stats.foreach { case (col, (mean, missingCount)) =>
  println(f"Column: $col, Missing values: $missingCount, Replaced with mean:$mean%.2f")
}

val cleanedRows = allRows.map { row =>
  numericColumns.foldLeft(row) { (accRow, col) =>
    val value = accRow.getOrElse(col, "").trim
    val replaced = Try(value.toDouble).toOption match {
      case Some(_) => value // valid number
      case None => f"${stats(col)._1}%.2f" // replace with mean
    }
    accRow.updated(col, replaced)
  }
}

val outputFile = new File("all_stocks_5yr_cleaned.csv")
val writer = CSVWriter.open(outputFile)
val headers = cleanedRows.head.keys.toSeq
writer.writeRow(headers)
cleanedRows.foreach(row => writer.writeRow(headers.map(row)))
writer.close()
println("\nMissing values replaced and saved to all_stocks_5yr_cleaned.csv")
}
}

```

Output for practical 9

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Use

--- Missing Data Report ---
Column: open, Missing values: 11, Replaced with mean:83.02
Column: high, Missing values: 8, Replaced with mean:83.78

Missing values replaced and saved to all_stocks_5yr_cleaned.csv

Process finished with exit code 0
```

Code for practical 10

```
package practical10
```

```
import com.github.tototoshi.csv._
import java.io.File
```

```
object practical10 {
  def main(args: Array[String]): Unit = {
    val reader = CSVReader.open(new File("all_stocks_5yr.csv"))
    val data = reader.allWithHeaders()
    reader.close()

    val threshold = 300.0

    val filteredRows = data.filter { row =>
      row.get("close").exists(value => value.toDoubleOption.exists(_ > threshold))
    }

    println(s"\nTotal Rows with Close Price > $threshold: ${filteredRows.length}\n")

    filteredRows.foreach { row =>
      println(row.values.mkString(", "))
    }
  }
}
```

Output for practical 10

SHETH LUJ AND MV COLLEGE

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Users\Asus
```

```
Total Rows with Close Price > 300.0: 12006
```

```
ADS, 297.36, 295.46, 2015-01-26, 300.65, 432892, 301.18
ADS, 299.14, 297.68, 2015-04-10, 300.82, 295631, 300.99
ADS, 299.66, 299.01, 2015-04-13, 300.7, 388781, 302.47
ADS, 301.04, 297.24, 2015-04-14, 301.07, 444752, 301.97
ADS, 301.22, 298.89, 2015-04-15, 302.2, 411522, 303.39
ADS, 300.27, 298.962, 2015-04-21, 304.86, 539709, 305.69
ADS, 304.75, 302.72, 2015-04-22, 308.33, 428753, 309.11
ADS, 306.6, 305.65, 2015-04-23, 309.91, 434424, 312.0
ADS, 309.93, 305.9421, 2015-04-24, 307.44, 258658, 309.93
ADS, 308.21, 304.02, 2015-04-27, 304.69, 338992, 308.78
ADS, 303.0, 300.99, 2015-04-28, 302.38, 378699, 306.21
ADS, 301.27, 295.51, 2015-04-29, 300.63, 363171, 303.12
ADS, 299.77, 297.37, 2015-05-04, 300.56, 444372, 302.39
ADS, 300.0, 297.975, 2015-05-05, 300.69, 333121, 301.28
ADS, 301.62, 301.0, 2015-05-08, 303.3, 503623, 305.68
ADS, 302.34, 301.16, 2015-05-11, 301.38, 379039, 305.21
ADS, 297.0, 296.528, 2015-05-18, 300.48, 326584, 301.07
ADS, 301.0, 298.844, 2015-05-19, 300.91, 226378, 303.03
ADS, 296.98, 296.542, 2015-05-22, 300.31, 339624, 301.69
ADS, 297.98, 297.03, 2015-05-27, 302.28, 565077, 302.46
ADS, 301.01, 299.43, 2015-05-28, 300.07, 406181, 304.02
ADS, 300.98, 299.4, 2015-06-11, 300.63, 285803, 302.82
ADS, 300.0, 299.47, 2015-06-12, 301.55, 324618, 303.0
ADS, 298.85, 297.2835, 2015-06-18, 301.27, 405565, 301.95
ADS, 300.62, 298.81, 2015-06-19, 302.71, 421955, 303.27
ADS, 303.96, 301.68, 2015-06-22, 305.5, 416570, 307.1
ADS, 306.26, 302.675, 2015-06-23, 304.18, 324294, 306.87
ADS, 304.2, 301.24, 2015-06-24, 303.39, 337315, 305.5
ADS, 303.95, 302.405, 2015-06-25, 303.97, 459238, 306.1
```

S091 Shreyash Kadam